

T4SKFORCE PNB

Updated Installation Document

A comprehensive guide including full hardware and software details, supported OS, dependencies, and required libraries.

Written for absolute beginners — every command, every click, explained.

Field	Details
Project	Quantum Security Assessment Platform (T4SKFORCE PNB)
Source	GitHub repository (public/shared repo)
Modules	quantum_scanner, quantum_crypto_backend_structure, quantum-dashboard
Audience	First-time setup users, judges, evaluators — no prior experience assumed

Table of Contents

1. Before You Begin
2. Project Overview & Repository Structure
3. Hardware Requirements
4. Supported Operating Systems
5. Software Prerequisites (Install Once)
6. Full Dependency & Library List
- 7. Step-by-Step Installation Guide (Main Installation Commands)**
8. Full System Verification Checklist
9. Troubleshooting & Frequently Asked Questions
10. Complete System Architecture
11. Quick-Reference Command Cheat Sheet

1. Before You Begin

This guide assumes you have never set up a Python or Node.js project before. Every command below can be copy-pasted exactly as written into your terminal (Command Prompt / PowerShell on Windows, Terminal on macOS/Linux). Follow the sections in order — do not skip ahead.

- You will install three modules, one at a time: the Scanner, the Backend, and the Dashboard.
- Each module has its own folder inside the GitHub repository — there is no .zip file to extract.
- Each step tells you exactly what to type and exactly what output to expect, so you know if it worked.
- If something doesn't match the expected output, jump to Section 9 (Troubleshooting) before continuing.

Note: Keep three separate terminal windows open once everything is running — one for the Scanner, one for the Backend, and one for the Dashboard. Closing a terminal window stops that service.

2. Project Overview & Repository Structure

The project is distributed as a single GitHub repository containing three independent, ready-to-run modules. You do not need to download or extract anything — cloning the repository gives you all three folders directly.

T4SKFORCE_PNB (GitHub Repository)

```
├─ quantum_scanner/
  Layer-1 Asset Discovery & CBOM Generator — runs on port 5000

├─ quantum_crypto_backend_structure/
  Quantum Risk Analysis & Scoring Engine — runs on port 8000

└─ quantum-dashboard/
  Web Dashboard (Frontend) — runs on port 3000
```

All three folders live directly inside the repository root.
There is no .zip file to extract — simply "git clone" the repo and open it.

Figure 1. GitHub repository structure — three folders at the repository root.

2.1 Module Responsibilities

Module	Purpose	Default Port
Quantum Scanner	Discovers assets and generates the Cryptographic Bill of Materials (CBOM)	5000
Quantum Backend	Performs quantum risk analysis, readiness scoring, AI insights and recommendation generation	8000
Quantum Dashboard	Visualizes scanner output and quantum analysis results	3000

2.2 Startup Order (Important)

The three modules must be started in this exact order, because each one depends on the one before it:

- 1. Quantum Scanner — starts first, has no dependency on the other two.
- 2. Quantum Backend — consumes data produced by the Scanner.

- 3. Quantum Dashboard — depends on both the Backend and Scanner APIs being reachable.

3. Hardware Requirements

The table below covers running all three modules together on a single laptop or desktop machine (recommended for local testing/demo). If you plan to run each module on a separate machine, the per-module table further below applies instead.

3.1 Combined Requirements (All 3 Modules on One Machine)

Component	Minimum	Recommended
CPU	Dual-core, 2.0 GHz (e.g. Intel i3 / equivalent)	Quad-core, 2.5 GHz+ (Intel i5/i7, AMD Ryzen 5/7, or Apple Silicon)
RAM	8 GB	16 GB or more
Disk Space (free)	5 GB free	10 GB+ free (SSD recommended)
Network	Active internet connection (for scanning targets, package downloads, and ngrok tunnels)	Stable broadband connection
Display	1366 x 768 or higher	1920 x 1080 (Full HD) or higher, for comfortable dashboard viewing

3.2 Per-Module Hardware Breakdown

Module	Minimum RAM	Minimum Disk	Notes
Quantum Scanner	2 GB available	1 GB free	Network-scanning tools (Nmap, SSLyze) benefit from a stable network connection.
Quantum Backend	2 GB available	1 GB free	Scoring and AI-inference workload is lightweight (classical ML model, not deep learning).
Quantum Dashboard	2 GB available	1 GB free	Standard Node.js/Next.js web app; RAM usage grows with browser tabs open, not the app itself.

4. Supported Operating Systems

Module	Supported OS	Notes
Quantum Scanner	Recommended: Ubuntu 22.04 LTS (or another supported Linux distribution)	Linux is the officially tested and recommended deployment platform for the Scanner. The Scanner may run on other operating systems (Windows, macOS) if all required dependencies (OpenSSL, Nmap, SSLyze, Python 3.10+) are correctly installed and available on the system PATH, but this configuration is not officially tested and is not guaranteed to behave identically. For the smoothest experience, use Ubuntu 22.04 LTS natively or via WSL2 on Windows.

Module	Supported OS	Notes
Quantum Backend	Windows 10/11, macOS 12+, Linux (any distribution)	Cross-platform — requires only Python 3.10+ and pip. No OS-specific dependencies.
Quantum Dashboard	Windows 10/11, macOS 12+, Linux (any distribution)	Cross-platform — requires only Node.js (LTS) and npm/yarn. No OS-specific dependencies.

Recommendation: If you are on Windows and want the smoothest Scanner setup (matching the officially tested environment), install WSL2 (Windows Subsystem for Linux) with an Ubuntu 22.04 LTS image, and run the Scanner module inside that Ubuntu environment. The Backend and Dashboard can still run directly on Windows.

5. Software Prerequisites (Install Once)

Install the following software before touching any of the three modules. You only need to do this once per machine.

Software	Required For	Minimum Version
Git	Cloning the GitHub repository	Any recent version
Python	Scanner + Backend runtime	3.10 or higher
pip	Python package manager (installed automatically with Python)	Latest
Node.js	Dashboard runtime	LTS release (18.x or higher)
npm (or yarn)	Dashboard package manager (installed automatically with Node.js)	Latest
OpenSSL	TLS handshake & certificate extraction (used by Scanner)	1.1.1 or higher
Nmap	Network discovery (used by Scanner)	7.9 or higher
SSLyze	TLS & cipher enumeration (used by Scanner)	Installed via pip
ngrok (optional)	Exposing local APIs publicly, so the dashboard can reach them if not all on one machine	Latest free tier
Code editor (optional)	Viewing/editing configuration files such as apiConfig.ts	VS Code recommended

5.1 Installing Git

Windows: download and run the installer from git-scm.com, accepting the default options.

macOS:

```
brew install git
# or install Xcode Command Line Tools:
xcode-select --install
```

Ubuntu / Debian Linux:

```
sudo apt update
sudo apt install git -y
```

Verify installation on any OS:

```
git --version
```

Note: You should see something like git version 2.x.x. If you see “command not found”, revisit the install step for your OS.

5.2 Installing Python 3.10+

Windows: download the installer from python.org/downloads. During installation, tick the checkbox “Add Python to PATH” before clicking Install — this step is easy to miss and causes most beginner errors.

macOS:

```
brew install python@3.10
```

Ubuntu / Debian Linux:

```
sudo apt update
sudo apt install python3 python3-pip python3-venv -y
```

Verify installation:

```
python3 --version
pip3 --version
```

Note: On Windows, the commands are usually python and pip instead of python3 and pip3. If python3 is not recognized on Windows, use python.

5.3 Installing Node.js (LTS)

Windows / macOS: download the LTS installer from nodejs.org and run it with default options.

Ubuntu / Debian Linux:

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt install nodejs -y
```

Verify installation:

```
node --version
npm --version
```

5.4 Installing OpenSSL

Windows: OpenSSL is bundled with Git for Windows (Git Bash). Alternatively install via chocolatey:

```
choco install openssl
```

macOS:

```
brew install openssl
```

Ubuntu / Debian Linux (usually pre-installed):

```
sudo apt install openssl -y
```

Verify:

```
openssl version
```

5.5 Installing Nmap

Windows: download the installer from nmap.org/download.html.

macOS:

```
brew install nmap
```

Ubuntu / Debian Linux:

```
sudo apt install nmap -y
```

Verify:

```
nmap --version
```

5.6 Installing SSLyze

SSLyze is a Python package and is installed automatically later when you run `pip install -r requirements.txt` inside the Scanner folder. You can also verify or install it manually on any OS:

```
pip install sslyze
```

Verify:

```
sslyze --help
```

5.7 Installing ngrok (Optional but Recommended)

Download ngrok from ngrok.com/download for your OS, unzip it, and place the executable somewhere on your system PATH (or run it from the download folder). Sign up for a free account and connect your auth token as instructed on their site:

```
ngrok config add-authtoken <your-token-here>
```

Note: ngrok is only required if the Dashboard needs to reach the Scanner/Backend over the internet (e.g., different machines, or sharing a live demo link). If everything runs on one machine for local testing, you can skip ngrok and use localhost URLs instead.

6. Full Dependency & Library List

Each module ships with its own dependency manifest (`requirements.txt` for Python modules, `package.json` for the Dashboard). Running the install command inside each folder (explained in Section 7) automatically downloads every

required library — you never need to install libraries one by one. The tables below list the core libraries and tools each module relies on, so you know what is being installed and why.

6.1 Quantum Scanner — Dependencies

Library / Tool	Type	Purpose
FastAPI	Python package	REST API framework that exposes the /scan endpoint
Uvicorn	Python package	ASGI server that runs the FastAPI application
SSLyze	Python package	TLS and cipher-suite enumeration
cryptography	Python package	Certificate parsing and cryptographic key analysis
requests	Python package	Outbound HTTP calls made during scanning
OpenSSL	System tool	Low-level TLS handshake and certificate extraction
Nmap	System tool	Network / port discovery

Note: Install all of the above in one step by running `pip install -r requirements.txt` inside the `quantum_scanner` folder. The manual command `pip install fastapi uvicorn ssllyze cryptography requests` is provided as a fallback only if `requirements.txt` is missing or incomplete.

6.2 Quantum Backend — Dependencies

Library / Tool	Type	Purpose
FastAPI	Python package	REST API framework that exposes the analysis / scoring endpoints
Uvicorn	Python package	ASGI server that runs the FastAPI application
(additional packages)	Python packages	Any further scoring, AI-insight, and data-handling libraries are pinned in <code>quantum_crypto_backend_structure/requirements.txt</code> and installed automatically

Note: Install all backend dependencies in one step by running `pip install -r requirements.txt` inside the `quantum_crypto_backend_structure` folder.

6.3 Quantum Dashboard — Dependencies

Library / Tool	Type	Purpose
Next.js	Node package	React-based web framework that serves the dashboard
React	Node package	UI component library used to build the dashboard views
(additional packages)	Node packages	Charting, styling, and utility libraries are pinned in <code>quantum-dashboard/package.json</code> and installed automatically

Note: Install all dashboard dependencies in one step by running `npm install` (or `yarn install`) inside the `quantum-dashboard` folder.

7. Step-by-Step Installation Guide

Make sure you have completed every install in Section 5 before starting here. Open a terminal window to begin.

7.1 Get the Code from GitHub

Choose a location on your computer for the project (e.g. your Desktop or a “Projects” folder), then run:

```
cd Desktop
git clone <the-github-repository-url>
cd <repository-folder-name>
ls          (Mac/Linux)  OR  dir  (Windows)
```

You should see exactly three folders listed:

```
quantum_scanner
quantum_crypto_backend_structure
quantum-dashboard
```

Note: Replace <the-github-repository-url> with the actual link you were given, e.g. <https://github.com/<org>/<repo>.git>. Replace <repository-folder-name> with whatever folder name git created (usually the repository name).

7.2 Part 1 — Deploy the Quantum Scanner (Layer-1)

Step 1 — Enter the Scanner Folder

```
cd quantum_scanner
```

Step 2 — Create a Virtual Environment

A virtual environment keeps this project's Python packages separate from everything else on your computer.

Linux / macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Windows (Command Prompt or PowerShell):

```
python -m venv venv
venv\Scripts\activate
```

Note: When activated correctly, you will see (venv) at the start of your terminal prompt line. If PowerShell blocks activation with a security/execution-policy error, see Section 9.

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

If any package fails to install automatically, install the core packages manually as a fallback:

```
pip install fastapi uvicorn sslyze cryptography requests
```

Step 4 — Verify External Security Tools Are Installed

```
openssl version
nmap --version
sslyze --help
```

Each command should print a version number or help text, not an error.

Step 5 — Verify Project Structure

Confirm the `layer1` folder contains files similar to these (you don't need to open them, just confirm they exist):

```
layer1/
  scanner_api.py
  main.py
  input_manager.py
  service_discovery.py
  tls_extractor.py
  cert_parser.py
  cipher_extractor.py
  key_analyzer.py
  cbom_builder.py
  ...
```

Step 6 — Start the Scanner API

```
uvicorn scanner_api:app --host 0.0.0.0 --port 5000
```

Expected output:

```
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:5000
```

Note: Leave this terminal window open and running. Open a NEW terminal window for the next steps.

Step 7 — Expose the Scanner Publicly (Optional)

Only needed if the Dashboard will run on a different machine/network. In a new terminal:

```
ngrok http 5000
```

ngrok will print a public URL such as: `https://abcd1234.ngrok-free.app` — copy this down, you will need it in Section 7.4.

Step 8 — Verify the Scanner Is Running

Open a web browser and go to:

```
http://localhost:5000/docs
```

You should see an interactive Swagger UI page listing the API's endpoints.

Step 9 — Run a Test Scan

On the Swagger UI page, open the POST `/scan` endpoint, click “Try it out”, and submit this example request body:

```
{
  "url": "google.com"
}
```

The scanner will perform target normalization, DNS resolution, port discovery, TLS handshake, certificate parsing, cipher enumeration, key analysis, API discovery, VPN detection, asset classification, and CBOM generation.

Step 10 — Expected Output

The response is a Cryptographic Bill of Materials (CBOM) containing scan metadata, asset information, TLS details, certificate details, cipher inventory, discovered APIs, services, and VPN endpoints. If you see this JSON response, the Scanner module is fully working.

7.3 Part 2 — Deploy the Quantum Backend

Step 1 — Enter the Backend Folder

Open a new terminal window (leave the Scanner terminal running) and navigate to the repository root, then into the backend folder:

```
cd quantum_crypto_backend_structure
```

Step 2 — Create a Virtual Environment

Linux / macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Windows:

```
python -m venv venv
venv\Scripts\activate
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

Step 4 — Start the Backend Server

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Expected output:

```
INFO:      Uvicorn running on http://0.0.0.0:8000
```

Note: Keep this terminal window running too. You should now have two terminals open: Scanner (port 5000) and Backend (port 8000).

Step 5 — Expose the Backend Publicly (Optional)

In another new terminal:

```
ngrok http 8000
```

Copy the resulting public URL (e.g. <https://abcd1234.ngrok-free.app>) — needed in Section 7.4.

Step 6 — Verify the Backend Is Running

```
http://localhost:8000/docs
```

A Swagger UI page should appear.

Step 7 — Run a Test Analysis

Open the Scan endpoint and submit an example request:

```
{  
  "target": "www.hdfc.bank.in"  
}
```

Step 8 — Expected Output

The backend returns an Executive Summary, Asset Profile, Cryptographic Bill of Materials, Security Assessment, Quantum Risk Analysis, HNDL Probability, Crypto Agility, Quantum Readiness Score, AI Risk Insights, PQC Migration Recommendations, Quantum Security Certificate, and Visualization Data. If you see this response, the Backend module is fully working.

7.4 Part 3 — Deploy the Quantum Dashboard

Step 1 — Enter the Dashboard Folder

Open a third new terminal window and navigate to the repository root, then into the dashboard folder:

```
cd quantum-dashboard
```

Step 2 — Install Dependencies

```
npm install
```

or, if you prefer yarn:

```
yarn install
```

Step 3 — Configure API Endpoints

Open the file `app/config/apiConfig.ts` in a text editor (VS Code or any editor). Update the Scanner and Backend URLs:

- If everything is running on the same machine, use the localhost addresses:

```
SCANNER_API=http://localhost:5000  
BACKEND_API=http://localhost:8000
```

- If using ngrok (different machines / public demo), use the ngrok URLs you copied earlier:

```
SCANNER_API=https://xxxx.ngrok-free.app
BACKEND_API=https://yyyy.ngrok-free.app
```

Note: If you restart ngrok, the URL changes every time (on the free plan). Remember to update `apiConfig.ts` again with the new URL before starting the Dashboard.

Step 4 — Verify CORS Configuration

Make sure the Backend accepts requests from the Dashboard's domain (localhost:3000, or your deployed domain). This is normally already configured in the backend module; if you see CORS errors in the browser console, see Section 9.

Step 5 — Start the Development Server

```
npm run dev
```

Open your browser and go to:

```
http://localhost:3000
```

Confirm that: the dashboard UI loads successfully, the Scanner API is reachable, and the Backend API responds correctly (try running a scan from the dashboard UI itself).

Step 6 — Production Build (Optional)

```
npm run build
npm run start
```

Step 7 — Managed Hosting (Optional)

Instead of running the Dashboard locally, you can deploy it directly to a hosting platform such as Vercel, Netlify, or AWS Amplify. These platforms automatically detect the Next.js project and handle the build and deployment process for you — just connect your GitHub repository and select the `quantum-dashboard` folder as the project root.

8. Full System Verification Checklist

Go through this checklist after completing all three deployments to confirm everything is working end-to-end.

#	Check	Expected Result
1	Scanner Swagger UI loads at <code>:5000/docs</code>	Interactive API docs page appears
2	Backend Swagger UI loads at <code>:8000/docs</code>	Interactive API docs page appears
3	Dashboard loads at <code>:3000</code>	Web UI appears with no blank/error screen
4	Test scan via Scanner <code>/scan</code> endpoint	Returns a CBOM JSON response
5	Test analysis via Backend <code>/scan</code> endpoint	Returns full risk analysis JSON response
6	Run a scan from the Dashboard UI	Dashboard displays live results without connection errors
7	<code>apiConfig.ts</code> URLs match active Scanner/Backend URLs	No “network error” / “failed to fetch” messages in browser console

9. Troubleshooting & Frequently Asked Questions

“python” or “python3” is not recognized

Python was not added to your system PATH during installation. On Windows, re-run the Python installer and check “Add Python to PATH”, or reinstall. On Mac/Linux, confirm installation with which python3.

“pip” is not recognized

Use the module form instead, which always works if Python itself is installed correctly:

```
python -m pip install -r requirements.txt
```

PowerShell blocks venv activation (“running scripts is disabled on this system”)

Run PowerShell as Administrator and execute the following once, then retry activation:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

“Address already in use” / “port already in use”

Another program is already using that port. Either stop the other program, or start this module on a different port, e.g.:

```
uvicorn scanner_api:app --host 0.0.0.0 --port 5001
```

(Remember to also update apiConfig.ts to match the new port if you change one.)

“ngrok: command not found”

The ngrok executable is not on your PATH. Either move it into a folder that is on your PATH, or run it using its full file path, e.g. ./ngrok http 5000 from inside the folder where you downloaded it.

Dashboard shows a CORS error in the browser console

This means the Backend is rejecting requests from the Dashboard's origin. Confirm the Backend's CORS settings allow http://localhost:3000 (or your ngrok/deployed dashboard URL), and confirm apiConfig.ts points to the correct, currently-running Backend URL.

Swagger UI page won't load at all

Confirm the corresponding terminal window still shows the server running (no crash/error text). Confirm you typed the exact URL (http://localhost:PORT/docs). Check that a firewall isn't blocking the port.

Scan results look empty or incomplete

Confirm OpenSSL, Nmap, and SSLyze are all correctly installed and are reachable from the same terminal/environment where you started the Scanner (Step 4 of Section 7.2). Re-run the verification commands in that section.

ngrok URL keeps changing

This is expected behavior on ngrok's free plan — a new URL is issued every time you restart the tunnel. Update apiConfig.ts each time this happens, or upgrade to a paid ngrok plan for a fixed subdomain.

10. Complete System Architecture

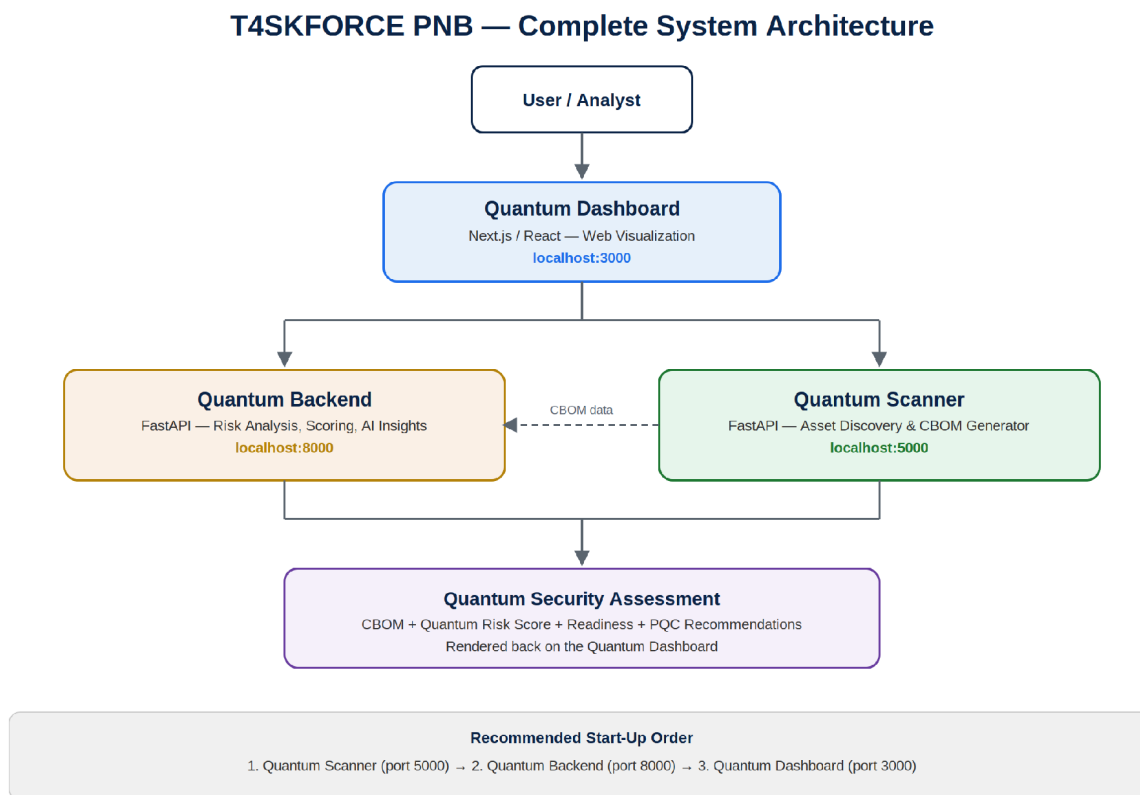


Figure 2. Complete system architecture — modules, ports, and data flow.

The Dashboard (port 3000) is the single entry point for the user. It calls the Backend (port 8000) for risk analysis and scoring, and the Backend in turn consumes the Cryptographic Bill of Materials produced by the Scanner (port 5000). The combined output — CBOM, quantum risk score, readiness score, and PQC recommendations — flows back up and is rendered on the Dashboard.

11. Quick-Reference Command Cheat Sheet

Quantum Scanner (port 5000)

```
cd quantum_scanner
python3 -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn scanner_api:app --host 0.0.0.0 --port 5000
```

Quantum Backend (port 8000)

```
cd quantum_crypto_backend_structure
python3 -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Quantum Dashboard (port 3000)

```
cd quantum-dashboard  
npm install  
# edit app/config/apiConfig.ts with Scanner + Backend URLs  
npm run dev
```

Note: Save this section for future reference — it contains every command needed to bring the full system back up after a restart.